Tomato Leaf Disease

Complete Project Implementation

Technology stack, code structure, configuration, execution flow, evaluation results, and screenshots

Project team

Issac Jacob Abraham and Naman Sharma

Project review date: 30 October 2026

Repository

GitHub repository link: https://github.com/namanshar6/tomato-leaf-disease-decision-support

The implementation package includes source code, the trained model, class mapping, evaluation artifacts, screenshots, and launch instructions. The training image dataset is not included.

1. Project summary

This project provides a Streamlit web app for tomato leaf disease classification and treatment decision support. A MobileNetV2 transfer-learning model classifies an uploaded tomato leaf photo into one of ten PlantVillage tomato classes. An explainable prototype layer adds an uncertainty gate, a simple visual severity estimate, optional environmental context, rule-based risk prioritization, general treatment categories, and post-treatment feedback logging.

The disease classifier is a trained model. The severity estimate and risk score are heuristics; the feedback log does not train the model. The app is intended as a first-line educational aid and not as a substitute for diagnosis by an agricultural expert.

2. Technology stack and programming languages

Area	Technology	Use
Programming language	Python 3	App, data preparation, model training, and evaluation.
Web interface	Streamlit	Browser-based upload, prediction, context forms, and results.
Deep learning	TensorFlow / Keras; MobileNetV2	Transfer-learning classifier and inference.
Image processing	Pillow, NumPy	RGB conversion, resize, preprocessing, and visual color proxy.
Metrics and plots	scikit-learn, Matplotlib, Seaborn	Classification metrics and confusion matrix.
Model artifact	HDF5 (.h5), JSON	Trained weights/model and class index mapping.

3. Project files

File	Implementation role
app.py	Streamlit UI, image inference, saved evaluation view, live mini-evaluation, context inputs, recommendation output, and feedback form.
train.py	Builds MobileNetV2 transfer-learning model and trains the classification head, then fine-tunes the final 30 backbone layers.
evaluate.py	Runs inference on data/val, calculates accuracy and macro precision/recall/F1, and saves confusion_matrix.png and eval_metrics.json.
data_prep.py	Finds tomato class folders in a source dataset and creates train/validation directories.
context_engine.py	Severity proxy, transparent risk score, contextual priority, follow-up guidance, and JSONL feedback append.
treatment_recommendations.py	Class descriptions and general treatment/prevention lookup text.
Launch Tomato Leaf App.bat and .lnk	Windows launcher and double-click shortcut; creates .venv, installs requirements, and starts Streamlit.

4. Configuration and execution

Runtime configuration

Setting	Configured value
Input image	JPG or PNG; converted to RGB and resized to 224 x 224.
Model path	model/tomato_disease_model.h5
Class mapping	model/class_indices.json
Dataset layout for training/evaluation	data/train//... and data/val//...
Training	Default 15 frozen-backbone epochs plus 5 fine-tuning epochs; batch size 32.
Uncertainty gate	Withhold recommendations if top confidence is below 70%, or top-two gap is below 12 percentage points.
Python 3.14 setup	Requirements specify TensorFlow 2.22.0rc0 or newer for Python 3.14; the release candidate model compatibility should be confirmed on the target computer.

Launch procedure

1. Double-click Tomato Leaf App.lnk in the project folder. 2. On first launch, the batch file creates a local virtual environment and installs requirements.txt. 3. Keep the launcher window open while using the browser app. 4. Upload a clear single tomato-leaf JPG or PNG for a prediction.

The training dataset is not bundled. To reproduce the original full validation evaluation, obtain the PlantVillage tomato images, run data_prep.py, and create data/val before running evaluate.py.

5. Model architecture

MobileNetV2 pretrained on ImageNet forms the feature backbone. The classification head uses global average pooling, dropout, a 128-unit ReLU dense layer, another dropout layer, and a ten-class softmax. Training initially freezes the backbone and then unfreezes its final 30 layers at a low learning rate. Augmentation uses rotation, shifts, shear, zoom, and horizontal flips.

6. Runtime prediction and decision flow

After image preprocessing, the model returns ten class scores. If confidence is below 70% or the leading class is within 12 percentage points of the runner-up, the app asks for another image and withholds treatment guidance. Otherwise, a color heuristic estimates visible severity from green-ish pixels; the user can adjust Low, Moderate, or High. Optional environmental measurements then feed a transparent risk heuristic.

7. Context, recommendation, and feedback logic

The app collects optional temperature, relative humidity, soil moisture, leaf wetness, rainfall, crop stage, and previous treatment response. The risk heuristic starts at 15 for non-healthy predictions. Moderate severity adds 12 and High adds 25. For moisture-associated classes, high humidity adds 15, long leaf wetness adds 20, recent rainfall adds 10, and 15-25 C adds 8. Spider mite prediction with humidity below 45% adds 12. Low soil moisture adds a note but no points. Scores are capped at 100; bands are Low below 35, Moderate from 35 to 64, and High at 65 or above.

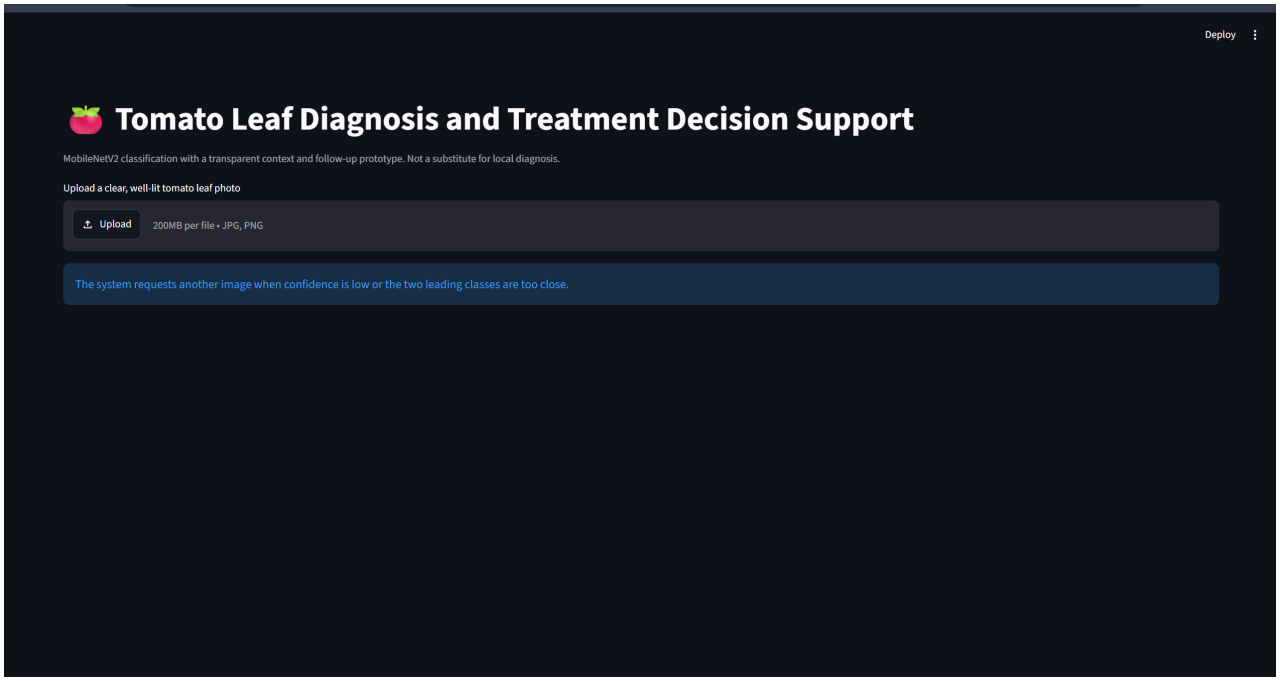
For confident disease results, the app displays the class lookup guidance and general cultural, biological, and chemical options. The user is told to follow local product registration and label directions and confirm with an extension officer. The feedback form records the user-reported follow-up severity, response, optional treatment description, and timestamp in model/treatment_feedback.jsonl. This record does not retrain the classifier or prove treatment effect.

8. Evaluation and output screenshots

The bundled evaluation report states validation metrics from model/eval_metrics.json. These are saved validation results, not a newly run field test. The live mini-evaluation added to the app can accept a small number of user-labeled images and generate a fresh demonstration confusion matrix during a presentation; its results must be labeled as a small demo subset.

Metric	Saved validation result
Accuracy	92.03%
Macro precision	91.03%
Macro recall	90.18%
Macro F1-score	90.04%

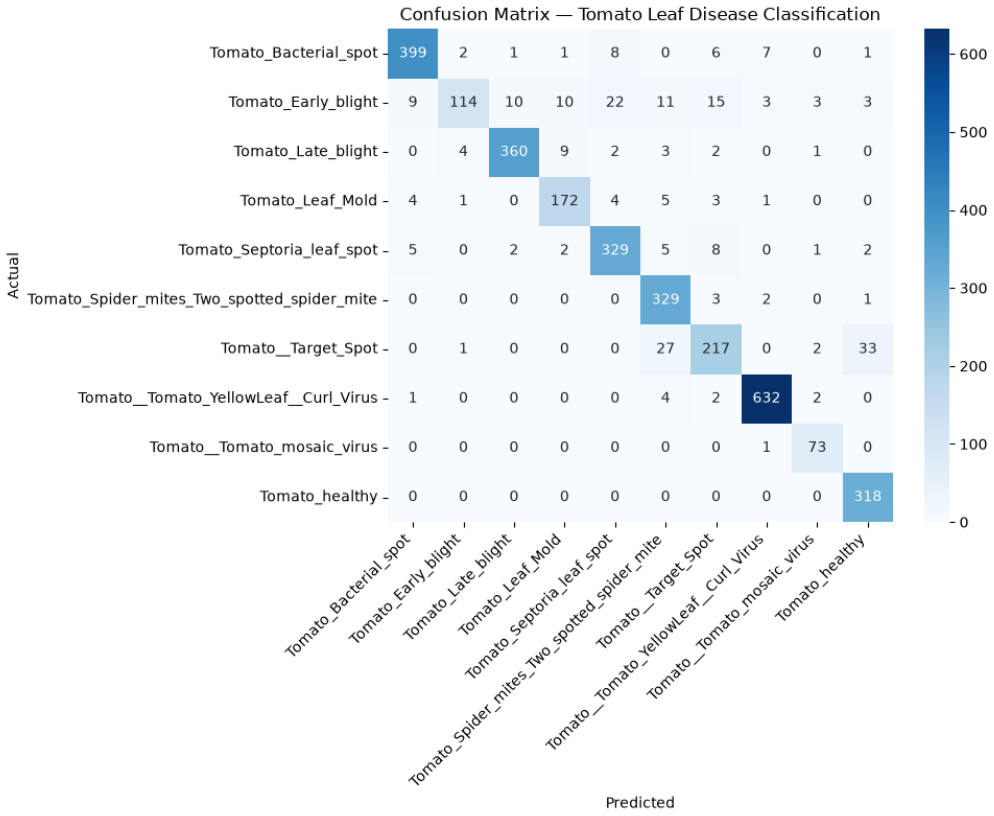
App interface screenshot



Screenshot of the running app upload screen. The app accepts a single tomato leaf image for diagnosis.

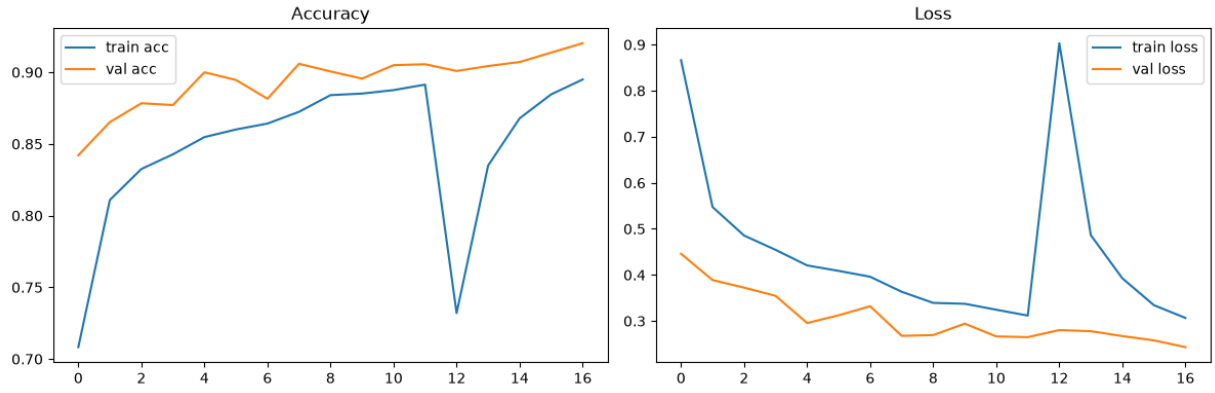
9. Evaluation output and live demonstration

Confusion matrix



Saved confusion matrix generated earlier by evaluate.py on the validation split.

Training curves



Accuracy and loss history included with the trained model package.

10. Live demonstration steps

To demonstrate code execution in front of the reviewer, run the app from its shortcut and open the Review demo expander.

Option A: Show the saved full validation results

Expand Review demo: saved validation results. The app displays saved metrics and the confusion matrix produced earlier by evaluate.py. Explain that the full validation image dataset is not included in this package, so these artifacts are not recalculated in the current session.

Option B: Generate a new small confusion matrix during the presentation

Use Run a live mini-evaluation. Upload a few leaf photos, assign each its known disease class, and select Run evaluation and generate matrix. The running model predicts each photo and the app calculates a new confusion matrix and metrics from the uploaded subset. The labels must be correct. Report this as a live demonstration subset, not as the full validation benchmark.

11. Limitations and reproducibility

The training dataset is absent from the submitted folder; only the trained model, class-index mapping, saved metrics, plots, and source code are packaged. PlantVillage images use controlled backgrounds, and field generalization has not been established. The severity image-color proxy and risk score are not trained or validated prediction models. Treatment feedback is a local log only. TensorFlow 2.22.0rc0 is a release candidate used to support Python 3.14; compatibility with the existing H5 model must be verified on the presentation machine.

12. GitHub repository and review access

GitHub repository URL: https://github.com/namanshar6/tomato-leaf-disease-decision-support. This repository is public, so the instructor can review all source files and report artifacts using the link. Do not upload data secrets, personal credentials, or the local .venv folder.

13. Submission checklist

- Use the GitHub URL included above. - Confirm that the public repository opens for the instructor. - Upload this final PDF to VTOP and Moodle. - Include the full project source, trained model, README, requirements, and screenshots in the repository. - State clearly that the raw training dataset is not included.